

Практикум 1. Настройка среды Docs as Code

Цель: Настроить базовую среду Docs as Code: Git, GitHub и генератор документации MkDocs или Docusaurus.

Что понадобится

- Git;
- аккаунт GitHub;
- любой редактор кода, например Visual Studio Code;
- Python для MkDocs **или** Node.js для Docusaurus.

Рекомендуемый вариант — MkDocs.

Задание

Необходимо:

- установить и настроить Git;
- создать репозиторий документации на GitHub;
- создать в нём простой документационный проект;
- настроить MkDocs или Docusaurus;
- подключить тему и базовый плагин/встроенную возможность;
- запустить документационный сайт локально;
- сохранить результат в GitHub.

Пошаговая инструкция

Шаг 1. Установите Git

Установите Git на свой компьютер.

После установки откройте терминал и выполните:

```
git --version
```

Если отображается версия Git — установка выполнена успешно.

Затем укажите имя и email:

```
git config --global user.name "Ваше Имя"  
git config --global user.email "your@email.com"
```

Email желательно использовать тот же, который связан с вашим аккаунтом GitHub.

Официальная инструкция по настройке Git:

Шаг 2. Создайте аккаунт GitHub

Если аккаунт GitHub уже есть — используйте его.

Если нет — зарегистрируйтесь на [GitHub](#).

Шаг 3. Создайте репозиторий

На GitHub создайте новый **публичный** репозиторий.

Название:

`docs-as-code`

Добавьте файл `README.md`.

GitHub позволяет создать репозиторий и сразу инициализировать его README-файлом. ([GitHub Docs](#))

В `README.md` напишите:

```
# Docs as Code
```

Практическая работа по настройке среды Docs as Code.

Автор: Имя Фамилия

Шаг 4. Склонируйте репозиторий

Скопируйте HTTPS-ссылку своего репозитория.

В терминале выполните:

```
git clone ССЫЛКА_НА_РЕПОЗИТОРИЙ
```

Например:

```
git clone https://github.com/username/docs-as-code.git
```

Перейдите в папку:

```
cd docs-as-code
```

Вариант А. MkDocs — рекомендуется

Шаг 5. Установите MkDocs

Для работы MkDocs потребуется Python и pip. MkDocs официально устанавливается через pip. ([MkDocs](#))

Выполните:

```
pip install mkdocs
```

Если команда pip не работает, попробуйте:

```
python -m pip install mkdocs
```

Проверьте установку:

```
mkdocs --version
```

Шаг 6. Создайте проект MkDocs

Находясь в папке репозитория, выполните:

```
mkdocs new .
```

После этого должна появиться примерно такая структура:

```
docs-as-code/
├── docs/
│   └── index.md
├── mkdocs.yml
└── README.md
```

Файл `mkdocs.yml` содержит настройки сайта, а папка `docs` — исходные Markdown-файлы документации. ([MkDocs](#))

Шаг 7. Добавьте страницу документации

Откройте:

```
docs/index.md
```

Добавьте небольшой текст:

```
# Моя документация
```

Это мой первый документационный проект в подходе Docs as Code.

```
## О проекте
```

Документация хранится в Git и создаётся с помощью MkDocs.

```
## Возможности
```

```
- Markdown
- Git
- GitHub
- MkDocs
```

Шаг 8. Настройте тему

Самый простой вариант — использовать встроенную тему `readthedocs`.

Откройте `mkdocs.yml` и приведите его примерно к следующему виду:

```
site_name: Docs as Code
```

```
theme:
  name: readthedocs
```

```
plugins:
  - search
```

MkDocs поддерживает встроенные темы, а плагин поиска `search` входит в MkDocs. ([MkDocs](#))

Таким образом, вы одновременно попробуете базовую настройку **темы и плагина**.

Шаг 9. Запустите сайт

В терминале выполните:

```
mkdocs serve
```

Откройте адрес, который появится в терминале.

Вы должны увидеть свой документационный сайт.

Сделайте **один скриншот работающего сайта** — он понадобится для сдачи

Вариант Б. Docusaurus

Вместо MkDocs можно использовать Docusaurus.

Для этого потребуется установленный Node.js.

Создайте проект:

```
npx create-docusaurus@latest my-website classic
```

Шаблон `classic` уже содержит базовую структуру документационного сайта и необходимые компоненты. (docusaurus.io)

Перейдите в папку проекта:

```
cd my-website
```

Запустите:

```
npm run start
```

Измените хотя бы одну страницу документации и заголовок сайта.

В Docusaurus базовые возможности документации подключаются через preset `classic`, поэтому отдельно устанавливать сторонний плагин в этой работе не требуется.

Сделайте **один скриншот работающего сайта**.

Шаг 10. Сохраните результат в Git

После выполнения работы добавьте изменения:

```
git add .
```

Создайте commit:

```
git commit -m "Create documentation project"
```

Отправьте изменения на GitHub:

```
git push
```

Откройте свой репозиторий на GitHub и убедитесь, что файлы появились.

Проверьте себя

Перед сдачей убедитесь, что:

- Git установлен и работает;
- создан публичный репозиторий `docs-as-code`;

- в репозитории есть README;
- установлен и настроен MkDocs **или** Docusaurus;
- есть хотя бы одна страница документации;
- настроена тема;
- подключён плагин или базовая возможность генератора;
- документационный сайт запускается;
- изменения отправлены в GitHub.

Что сдавать

Пришлите преподавателю **одну ссылку на публичный GitHub-репозиторий**.

В корень репозитория дополнительно положите папку:

screenshots/

В неё добавьте **один скриншот работающего документационного сайта**.

Итоговая структура для MkDocs будет примерно такой:

```
docs-as-code/  
├── docs/  
│   └── index.md  
├── screenshots/  
│   └── site.png  
├── mkdocs.yml  
└── README.md
```

Таким образом, для сдачи необходимо прислать только:

Фамилия Имя — ссылка на GitHub-репозиторий.

Критерии проверки

Зачёт, если:

- репозиторий открывается;
- присутствуют файлы документационного проекта;
- есть Markdown-страница с собственным содержанием;
- настроена тема и базовый плагин/возможность;
- есть скриншот работающего сайта;
- в GitHub виден хотя бы один commit с выполненной работой.

Главное в этой работе — самостоятельно пройти полный минимальный путь:

Git → GitHub → документационный проект → настройка → запуск → commit → push.